

Barren Plateaus and Trainability Issues

QML

Contents

1	Overview	3
2	What Is a Barren Plateau?	3
2.1	Formal Definition	3
2.2	Why This Is Devastating	3
3	The Original Result: McClean et al. 2018	4
3.1	Setup	4
3.2	The Key Theorem	4
3.3	Proof Sketch	5
3.4	The 2-Design Condition	5
4	Cost Function Dependent Barren Plateaus	6
4.1	Global vs Local Cost Functions	6
5	Noise-Induced Barren Plateaus	7
5.1	The Result	7
5.2	Implications	7
6	Entanglement-Induced Barren Plateaus	7
6.1	The Connection	8
7	Strategies to Avoid Barren Plateaus	8
7.1	Strategy 1: Local Cost Functions	8
7.2	Strategy 2: Shallow Circuits	9
7.3	Strategy 3: Layer-wise Training	9
7.4	Strategy 4: Parameter Initialization	10
7.5	Strategy 5: Problem-Inspired Ansatzes	10
8	Connection to Kernel Concentration	11
8.1	The Problem	11
8.2	The Unified Picture	11

9	Trainability vs Expressibility Tradeoff	12
9.1	The Tradeoff	12
9.2	Tradeoff Diagram	12
10	The Lie Algebra Perspective	13
10.1	Dynamical Lie Algebra (DLA)	13
10.2	DLA Dimensions of Common Ansätze	13
11	Gradient Variance vs Number of Qubits	14
11.1	Exponential Decay of Gradient Variance	14
12	Practical Guidelines for Circuit Design	14
12.1	The Checklist	14
12.2	Decision Flowchart	15
13	Advanced Topics	15
13.1	Overparameterization and Lazy Training	15
13.2	Quantum Natural Gradient	16
13.3	Error Mitigation and Trainability	16
14	Open Questions	16
15	Summary	17

1 Overview

this is arguably the most important lecture in the entire course. barren plateaus r the central obstacle to scaling variational quantum algorithms, and understanding them is essential for anyone who wants to design practical quantum ML models.

the basic problem: when u train a parameterized quantum circuit by gradient descent, the gradients can vanish exponentially in the number of qubits. not polynomially — exponentially. this means that for a 100-qubit circuit, the gradient might be $\sim 2^{-100}$, which is indistinguishable from zero with any reasonable number of measurement shots.

this isnt just a theoretical concern — its the main reason why scaling VQAs to useful problem sizes has been so difficult. in this lecture, well understand exactly why barren plateaus happen, when they occur, and wt we can do about them.

the good news: we now understand barren plateaus well enough to avoid them in many cases. the bad news: avoiding them often means restricting the model in ways that might limit its power. this tension between trainability and expressiveness is the central challenge of variational QML.

2 What Is a Barren Plateau?

2.1 Formal Definition

Definition 2.1 (Barren Plateau). a parameterized quantum circuit $U(\theta)$ with cost function $C(\theta)$ exhibits a barren plateau if the variance of the cost function partial derivatives vanishes exponentially with the number of qubits n :

$$\text{Var}_{\theta} \left[\frac{\partial C}{\partial \theta_k} \right] \leq F(n) \in O\left(\frac{1}{b^n}\right) \quad (1)$$

for some $b > 1$, where the variance is taken over random parameter initializations θ .

wt does this mean in practice? if the gradient variance is $O(2^{-n})$, then to estimate the gradient to precision ϵ , u need:

$$N_{\text{shots}} = O\left(\frac{\text{Var}[\partial_k C]}{\epsilon^2}\right) = O\left(\frac{1}{\epsilon^2 \cdot 2^n}\right) \quad (2)$$

wait thats wrong — u actually need more shots when the signal is small. the number of measurements to distinguish the gradient from zero is:

$$N_{\text{shots}} = O\left(\frac{1}{(\partial_k C)^2}\right) = O(2^n) \quad (3)$$

bcs the gradient itself is typically $O(2^{-n/2})$ when the variance is $O(2^{-n})$.

Intuition. imagine trying to find the lowest point in a landscape thats almost perfectly flat, with tiny random bumps of height $\sim 2^{-n}$. u cant tell which direction is downhill bcs ur measurement noise is way larger than the actual slope. thats a barren plateau. the landscape isnt featureless — it has structure — but the structure is invisible to any polynomial-time optimization procedure.

2.2 Why This Is Devastating

the exponential scaling is wt makes barren plateaus qualitatively different from the vanishing gradient problem in classical deep learning. in classical NNs:

- gradients vanish polynomially with depth (for vanilla networks without skip connections)
- solutions exist: residual connections, batch normalization, careful initialization
- these solutions work well in practice

in quantum circuits:

- gradients vanish exponentially with width (number of qubits)
- no analog of residual connections (unitarity prevents additive shortcuts)
- the problem is fundamental, not just a practical inconvenience

Key Point. barren plateaus r not just “quantum vanishing gradients.” the exponential scaling makes them qualitatively worse. in a classical network with vanishing gradients, u can often still make progress (slowly). in a barren plateau, the gradient is literally smaller than ur measurement precision for any practical number of shots. ur optimizer is effectively taking random walks.

3 The Original Result: McClean et al. 2018

the foundational paper by mcclean, boixo, smelyanskiy, napp, and babbush (2018) established that random parameterized quantum circuits generically exhibit barren plateaus.

3.1 Setup

consider a parameterized circuit:

$$U(\theta) = U_L(\theta_L) \cdots U_2(\theta_2) \cdot U_1(\theta_1) \quad (4)$$

and a cost function:

$$C(\theta) = \langle 0 | U^\dagger(\theta) O U(\theta) | 0 \rangle \quad (5)$$

where O is some observable (hermitian operator).

the partial derivative with respect to parameter θ_k (assuming it appears in gate $U_k = e^{-i\theta_k P_k/2}$ for some pauli P_k) is:

$$\frac{\partial C}{\partial \theta_k} = \frac{i}{2} \langle 0 | U^\dagger(\theta) [U_{k+}^\dagger O U_{k+} - U_{k-}^\dagger O U_{k-}] U(\theta) | 0 \rangle \quad (6)$$

where we use the fact that the parameter shift rule gives:

$$\frac{\partial C}{\partial \theta_k} = \frac{1}{2} [C(\theta_k + \pi/2) - C(\theta_k - \pi/2)] \quad (7)$$

3.2 The Key Theorem

Theorem 3.1 (McClean et al. 2018 — Barren Plateau Theorem). let $U(\theta) = U_L \cdots U_1$ be a parameterized quantum circuit on n qubits. suppose the gates $\{U_l\}$ form a 2-design (i.e., the distribution of unitaries matches the Haar measure up to the second moment). then for any observable O and any parameter θ_k :

$$\mathbb{E}_\theta \left[\frac{\partial C}{\partial \theta_k} \right] = 0 \quad (8)$$

and

$$\text{Var}_\theta \left[\frac{\partial C}{\partial \theta_k} \right] \leq \frac{2\text{Tr}(\mathcal{O}^2)}{2^{2n} - 1} \quad (9)$$

which is $O(2^{-n})$ for any observable with polynomial $\text{Tr}(\mathcal{O}^2)$.

3.3 Proof Sketch

the proof uses the theory of unitary t -designs and haar integration. here r the key steps:

step 1: decompose the circuit. write $U(\theta) = U_R \cdot V_k(\theta_k) \cdot U_L$ where U_L contains all gates before gate k , V_k is the gate containing θ_k , and U_R contains all gates after.

step 2: compute the gradient.

$$\partial_k C = \frac{i}{2} \text{Tr} \left[\mathcal{O} U_R V'_k U_L |0\rangle \langle 0| U_L^\dagger V_k^\dagger U_R^\dagger \right] \quad (10)$$

where $V'_k = [P_k, V_k]/2$ involves the commutator with the generator.

step 3: average over U_L using 2-design property. when U_L forms a 2-design, we can use the haar integral formula:

$$\int dU U \rho U^\dagger \otimes U \sigma U^\dagger = \frac{\text{Tr}(\rho)\text{Tr}(\sigma) + \text{Tr}(\rho\sigma)}{d(d+1)} (\mathbb{I} + \text{SWAP}) + \dots \quad (11)$$

where $d = 2^n$.

step 4: average over U_R similarly. the combination of averaging over both U_L and U_R squeezes out all the structure from the circuit, leaving only dimension-dependent factors.

step 5: bound the variance. after integrating, the variance picks up a factor of $1/d^2 = 1/4^n$:

$$\text{Var}[\partial_k C] = O\left(\frac{\|\mathcal{O}\|_F^2}{4^n}\right) \quad (12)$$

where $\|\mathcal{O}\|_F^2 = \text{Tr}(\mathcal{O}^2)$.

Intuition. the proof boils down to: if ur circuit is expressive enough to look like a random unitary, then the cost landscape becomes flat bcs a random unitary “scrambles” all information. the observable $\mathcal{O}$ gets diluted across the entire 2^n -dimensional hilbert space, and the gradient signal drowns in the exponentially large space.

3.4 The 2-Design Condition

a key assumption is that the circuit forms a 2-design. when does this happen?

Definition 3.1 (Unitary 2-Design). an ensemble of unitaries $\{U_i\}$ forms a 2-design if:

$$\mathbb{E}_i \left[U_i^{\otimes 2} \rho (U_i^\dagger)^{\otimes 2} \right] = \int_{\text{Haar}} U^{\otimes 2} \rho (U^\dagger)^{\otimes 2} dU$$

for all operators ρ on $(\mathbb{C}^d)^{\otimes 2}$.

in practice, many common ansatz architectures converge to 2-designs quickly:

- random circuits with CNOT + single-qubit rotations: becomes 2-design in $O(n)$ depth

- hardware-efficient ansatz (HEA): typically becomes 2-design within a few layers
- the more expressive ur circuit, the faster it converges to 2-design, and the worse the barren plateau

Key Point. this is the fundamental irony: more expressiveness leads to worse trainability. if ur circuit can express everything, it can learn nothing (efficiently). this is the expressibility-trainability tradeoff, and its the most important concept in this lecture.

4 Cost Function Dependent Barren Plateaus

cerezo, sone, volkoff, cincio, and coles (2021) showed that barren plateaus depend critically on the cost function structure, not just the circuit.

4.1 Global vs Local Cost Functions

Definition 4.1 (Global Cost Function). a cost function $C = \text{Tr}[O\rho(\theta)]$ is global if the observable O acts non-trivially on all n qubits. example: $O = |\psi\rangle\langle\psi|$ for some target state $|\psi\rangle$ (state fidelity).

Definition 4.2 (Local Cost Function). a cost function is local if $O = \frac{1}{n} \sum_{j=1}^n O_j$ where each O_j acts on $O(1)$ qubits. example: $O = \frac{1}{n} \sum_j Z_j$ (average magnetization).

Theorem 4.1 (Cerezo et al. 2021 — Cost-Dependent Barren Plateaus). for a parameterized circuit forming a local 2-design:

- (i) global cost: $\text{Var}[\partial_k C_G] \in O(2^{-n})$ — exponential vanishing regardless of circuit depth
- (ii) local cost: $\text{Var}[\partial_k C_L] \in \Omega(2^{-2L})$ for a circuit of depth L on n qubits — the gradient variance depends on the depth, not the width

this is a huge deal. it says:

- for global cost functions, ur screwed no matter wt — barren plateaus kick in with system size n
- for local cost functions, u can avoid barren plateaus by keeping the circuit shallow! the gradient variance depends on depth L , so as long as $L \in O(\text{poly}(\log n))$, ur fine

Example 4.1. consider two cost functions for state preparation:

$$C_G = 1 - |\langle \psi_{\text{target}} | U(\theta) | 0 \rangle|^2 \quad (\text{global — fidelity}) \quad (13)$$

$$C_L = 1 - \frac{1}{n} \sum_{j=1}^n \text{Tr}[\rho_j^{\text{target}} \rho_j(\theta)] \quad (\text{local — reduced state fidelity}) \quad (14)$$

where ρ_j is the reduced density matrix of qubit j . both cost functions vanish when $U(\theta)|0\rangle = |\psi_{\text{target}}\rangle$, but C_L is much more trainable bcs it only involves local measurements.

Intuition. why does locality help? a local cost function only “looks at” a few qubits at a time. changing a parameter in the circuit has a local effect that propagates through the circuit. if the circuit is shallow, this effect hasnt been scrambled across all qubits yet, so the gradient retains a non-vanishing signal. a global cost function requires all qubits to cooperate simultaneously, which becomes exponentially unlikely as n grows.

5 Noise-Induced Barren Plateaus

wang, larocca, czarnik, goyal, coles, and cerezo (2021) discovered smtg disturbing: hardware noise can independently cause barren plateaus, even when the noiseless circuit would be trainable.

5.1 The Result

Theorem 5.1 (Noise-Induced Barren Plateaus). consider a parameterized circuit with local depolarizing noise after each gate, with noise rate p . for any cost function (even local ones), the gradient variance satisfies:

$$\text{Var}[\partial_k C] \leq 2\|O\|^2 \cdot q^{2L} \quad (15)$$

where $q = 1 - p$ and L is the circuit depth. when $L \geq O(1/p)$, the gradients vanish exponentially:

$$\text{Var}[\partial_k C] \leq 2\|O\|^2 \cdot e^{-2pL} \quad (16)$$

this is independent of the cost function structure! even local cost functions — which r safe from “expressibility-induced” barren plateaus — suffer from noise-induced barren plateaus when the circuit is deep enough.

Key Point. noise-induced barren plateaus set a hard depth limit on trainable circuits:

$$L_{\max} \sim \frac{1}{p} \quad (17)$$

for typical NISQ error rates $p \sim 10^{-3}$ to 10^{-2} , this means $L_{\max} \sim 100$ to 1000 layers. sounds like a lot, but remember that each “layer” might involve just one gate, so this limits circuit depth to $O(100-1000)$ total gates.

Intuition. the mechanism is simple: noise pushes the quantum state toward the maximally mixed state $\mathbb{I}/2^n$. the maximally mixed state is a fixed point of any unitary, so all parameters become equally (un)important. the cost landscape flattens out bcs the state has lost all its quantum structure to decoherence.

5.2 Implications

this result creates a three-way tension:

1. expressibility wants deep circuits (more parameters, more expressiveness)
2. trainability wants shallow circuits (avoid barren plateaus)
3. noise tolerance wants even shallower circuits (avoid noise-induced BPs)

the viable region is the intersection of all three constraints:

$$L_{\text{viable}} \leq \min\left(L_{\text{trainable}}, \frac{1}{p}\right) \quad (18)$$

6 Entanglement-Induced Barren Plateaus

theres yet another mechanism for barren plateaus: too much entanglement.

6.1 The Connection

marrero, kieferová, and wiebe (2021) and ortiz marrero et al. showed that:

$$\text{Var}[\partial_k C] \leq \exp(-\Theta(S_{\text{ent}})) \quad (19)$$

where S_{ent} is the entanglement entropy across a bipartition of the qubits.

Definition 6.1 (Entanglement Entropy). for a bipartition of the system into subsystems A and B , the entanglement entropy is:

$$S_{\text{ent}}(\rho_A) = -\text{Tr}[\rho_A \log \rho_A] \quad (20)$$

where $\rho_A = \text{Tr}_B[|\psi\rangle\langle\psi|]$ is the reduced density matrix.

for a maximally entangled state on n qubits with equal bipartition: $S_{\text{ent}} = (n/2) \log 2$. so maximal entanglement implies $\text{Var}[\partial_k C] \leq e^{-\Theta(n)}$ — exponential vanishing.

Intuition. too much entanglement scrambles information across the system, making local changes (parameter updates) invisible to local measurements. its the same mechanism as in black hole physics: information thrown into a black hole gets scrambled across the entire event horizon. in a barren plateau, ur parameters r scrambled across the entire hilbert space.

Key Point. there r now at least four known mechanisms for barren plateaus:

- (i) expressibility (2-design circuits) — mcclean et al. 2018
- (ii) global cost functions — cerezo et al. 2021
- (iii) hardware noise — wang et al. 2021
- (iv) excessive entanglement — marrero et al. 2021

they can act independently or reinforce each other.

7 Strategies to Avoid Barren Plateaus

now for the constructive part. how do we design circuits that r trainable?

7.1 Strategy 1: Local Cost Functions

as cerezo et al. showed, local cost functions have gradient variance that scales with circuit depth, not width:

$$\text{Var}[\partial_k C_L] \geq \frac{c}{L^2} \quad (21)$$

for shallow circuits ($L \ll n$), where c is a constant.

recipe: replace global observables with sums of local ones:

$$C_{\text{global}} = \text{Tr}[O_{\text{global}} \rho] \longrightarrow \quad (22)$$

$$C_{\text{local}} = \frac{1}{n} \sum_{j=1}^n \text{Tr}[O_j \rho] \quad (23)$$

where each O_j acts on $O(1)$ qubits.

Example 7.1. for the VQE energy minimization $\langle H \rangle$, the hamiltonian is typically already a sum of local terms:

$$H = \sum_{\alpha} c_{\alpha} P_{\alpha}$$

where P_{α} r tensor products of pauli operators (at most k -local). so VQE naturally has a local cost function, which is one reason it works better than other variational algorithms.

7.2 Strategy 2: Shallow Circuits

keep circuit depth logarithmic in the number of qubits:

$$L \in O(\log n) \implies \text{Var}[\partial_k C_L] \in \Omega\left(\frac{1}{\text{poly}(n)}\right) \quad (24)$$

this ensures polynomial (not exponential) gradient vanishing. combined with local cost functions, $O(\log n)$ depth is sufficient to avoid barren plateaus entirely.

the catch: $O(\log n)$ depth limits expressiveness. the number of parameters is $O(n \log n)$, and the circuit can only generate states with limited entanglement (area-law-like). for some problems this is enough; for others, u need more depth.

7.3 Strategy 3: Layer-wise Training

instead of training all parameters simultaneously, grow the circuit incrementally:

1. start with a single layer $U_1(\theta_1)$
2. train θ_1 to convergence (easy — shallow circuit, no BP)
3. freeze θ_1 , add layer $U_2(\theta_2)$
4. train θ_2 to convergence (slightly deeper, but still manageable)
5. repeat until desired depth

$$U(\theta) = U_L(\theta_L) \cdots U_2(\theta_2^*) \cdot U_1(\theta_1^*) \quad (25)$$

where θ_i^* denotes frozen (previously optimized) parameters.

Intuition. this works bcs each new layer starts “close to identity” (if initialized correctly) and only needs to learn a small correction to the already-trained circuit. the optimization landscape for the new layer is smooth bcs the previous layers have already found a good region of parameter space.

Key Point. layer-wise training avoids the BP problem for deep circuits but introduces a greedy bias: each layer is trained myopically, without considering how later layers might compensate. this can lead to suboptimal solutions. a common compromise: layer-wise pre-training followed by global fine-tuning with small learning rate.

7.4 Strategy 4: Parameter Initialization

how u initialize parameters matters enormously.

identity initialization: set all parameters so that $U(\theta_0) \approx \mathbb{I}$:

$$\theta_0 = \vec{0} + \varepsilon \quad \text{where } \varepsilon \sim \mathcal{N}(0, \sigma^2), \sigma \ll 1 \quad (26)$$

this ensures the initial state is close to $|0\rangle$, which has zero entanglement and maximal structure. the optimization starts in a “non-barren” region and (hopefully) stays there.

small random initialization: similar idea but with small random perturbations:

$$\theta_{k,0} \sim \mathcal{U} \left[-\frac{a}{\sqrt{L}}, \frac{a}{\sqrt{L}} \right] \quad (27)$$

where a is a small constant and L is the circuit depth. the $1/\sqrt{L}$ scaling ensures that the cumulative effect of all layers is $O(1)$.

Example 7.2. comparison of initialization strategies for an 8-qubit hardware-efficient ansatz with 10 layers:

Initialization	Avg $ \nabla C $	Convergence
uniform $[0, 2\pi]$	$\sim 10^{-4}$	fails
uniform $[0, 0.1]$	$\sim 10^{-2}$	slow
identity + $\mathcal{N}(0, 0.01)$	$\sim 10^{-1}$	good

the difference between random and identity init can be 2-3 orders of magnitude in gradient size.

7.5 Strategy 5: Problem-Inspired Ansätze

the most effective strategy: use circuit architectures that respect the structure of ur problem.

QAOA (Quantum Approximate Optimization Algorithm): for combinatorial optimization, the circuit alternates between cost and mixer unitaries:

$$U(\gamma, \beta) = \prod_{p=1}^P e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \quad (28)$$

this has only $2P$ parameters regardless of system size, and the structure prevents BPs for shallow depth.

Hamiltonian Variational Ansatz (HVA): for physics problems, the circuit mirrors the structure of the hamiltonian:

$$U(\theta) = \prod_{l=1}^L \prod_{\alpha} e^{-i\theta_{l,\alpha} H_{\alpha}} \quad (29)$$

where $H = \sum_{\alpha} H_{\alpha}$ is the hamiltonian decomposition.

symmetry-preserving ansatz: if the target state has a symmetry (e.g., particle number conservation, spin symmetry), design the circuit to preserve it:

$$[U(\theta), S] = 0 \quad \forall \theta \quad (30)$$

where S is the symmetry operator. this restricts the effective hilbert space dimension and reduces BP severity.

Theorem 7.1 (Problem-Inspired Ansatzes Avoid Barren Plateaus). larocca, czarnik, coles, and cerezo (2022) showed that if the dynamical lie algebra (DLA) of the circuit generators has dimension $\dim(\mathfrak{g}) \in O(\text{poly}(n))$, then the circuit does not exhibit barren plateaus:

$$\text{Var}[\partial_k C] \in \Omega\left(\frac{1}{\text{poly}(n)}\right) \quad (31)$$

QAOA and HVA satisfy this condition bcs their generators span a polynomial-dimensional lie algebra.

Key Point. the hierarchy of ansatzes from worst to best for trainability:

1. hardware-efficient ansatz (HEA): generic, expressive, barren plateaus guaranteed for depth $> O(\log n)$
2. problem-agnostic but structured (e.g., alternating layers with limited entanglement): better, but still no guarantees
3. problem-inspired (QAOA, HVA, symmetry-preserving): provably BP-free for polynomial DLA dimension

8 Connection to Kernel Concentration

thanasilp, tangpanitanon, lemieux, jirawat, and coles (2024) unified barren plateaus with another phenomenon: kernel concentration.

8.1 The Problem

recall that quantum kernels are computed as:

$$k(x_i, x_j) = |\langle 0 | S^\dagger(x_i) S(x_j) | 0 \rangle|^2 \quad (32)$$

kernel concentration means that for large n , all kernel values converge to the same value:

$$k(x_i, x_j) \rightarrow \delta_{x_i, x_j} \quad (\text{diagonal dominance}) \quad (33)$$

or equivalently, the kernel matrix becomes approximately the identity matrix $K \approx I$.

8.2 The Unified Picture

Theorem 8.1 (Thanasilp et al. 2024). for quantum models based on circuits that form approximate 2-designs:

- (i) if the variational model exhibits barren plateaus, then the corresponding quantum kernel concentrates
- (ii) if the quantum kernel concentrates, the kernel-based model cannot learn (the kernel matrix carries no information about the data)
- (iii) both phenomena share the same root cause: excessive expressibility

this means the trainability problem isn't specific to variational methods — it's a fundamental limitation that also affects kernel methods. the quantum model, whether variational or kernel-based, becomes useless when the circuit is too expressive.

$$\underbrace{\text{Barren Plateaus}}_{\text{variational}} \iff \underbrace{\text{Kernel Concentration}}_{\text{kernel}} \iff \underbrace{\text{2-Design Convergence}}_{\text{root cause}} \quad (34)$$

Intuition. think of it this way: a circuit that forms a 2-design maps all inputs to “random-looking” states. random states in high-dimensional hilbert space are nearly orthogonal to each other ($\langle \psi | \phi \rangle \approx 0$), so the kernel gives no useful similarity information. simultaneously, the cost landscape is flat bcs the circuit is too random to have meaningful parameter dependence. same cause, same cure.

9 Trainability vs Expressibility Tradeoff

this is the central tension of variational QML. let's make it precise.

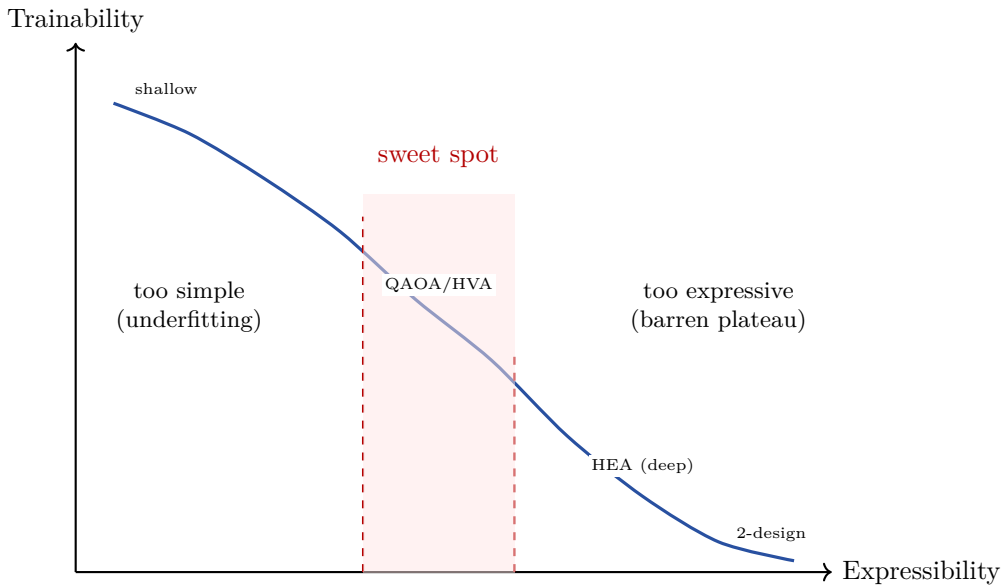
9.1 The Tradeoff

Definition 9.1 (Expressibility-Trainability Tradeoff). • expressibility: the set of states/functions the circuit can represent. more expressive = larger set = can fit more complex patterns

- trainability: ability to find good parameters via gradient-based optimization. more trainable = larger gradients = faster convergence

these two properties are fundamentally in tension: increasing expressibility (toward 2-design) decreases trainability (toward barren plateau).

9.2 Tradeoff Diagram



Key Point. the sweet spot is problem-dependent. for a simple problem, a less expressive (more trainable) circuit suffices. for a complex problem, you need more expressiveness and must accept

harder training. the art of QML circuit design is finding the right point on this curve for ur specific problem.

10 The Lie Algebra Perspective

larocca, sauvage, nardo, coles, and cerezo developed a beautiful framework connecting trainability to the algebraic structure of the circuit.

10.1 Dynamical Lie Algebra (DLA)

Definition 10.1 (Dynamical Lie Algebra). given a parameterized circuit with generators $\{iH_1, iH_2, \dots, iH_m\}$ (where $U_k(\theta_k) = e^{-i\theta_k H_k}$), the DLA is:

$$\mathfrak{g} = \text{span}_{\mathbb{R}} \langle iH_1, \dots, iH_m, [iH_j, iH_k], [[iH_j, iH_k], iH_l], \dots \rangle \quad (35)$$

the real vector space generated by the generators and all their nested commutators.

the dimension $\dim(\mathfrak{g})$ determines everything:

- $\dim(\mathfrak{g}) = 4^n - 1$ (maximal, $\mathfrak{su}(2^n)$): the circuit can reach any unitary. this is the 2-design regime. guaranteed barren plateau.
- $\dim(\mathfrak{g}) \in O(\text{poly}(n))$: the circuit is restricted to a polynomial-dimensional subspace of unitaries. no barren plateau.

Theorem 10.1 (Larocca et al. — DLA Dimension Determines Trainability). for a parameterized circuit with DLA $\mathfrak{g}$:

$$\text{Var}[\partial_k C] \in \Theta\left(\frac{1}{\dim(\mathfrak{g})}\right) \quad (36)$$

this is incredibly clean. the gradient variance scales inversely with the DLA dimension. polynomial DLA = polynomial gradients = trainable. exponential DLA = exponential vanishing = barren plateau.

10.2 DLA Dimensions of Common Ansatzes

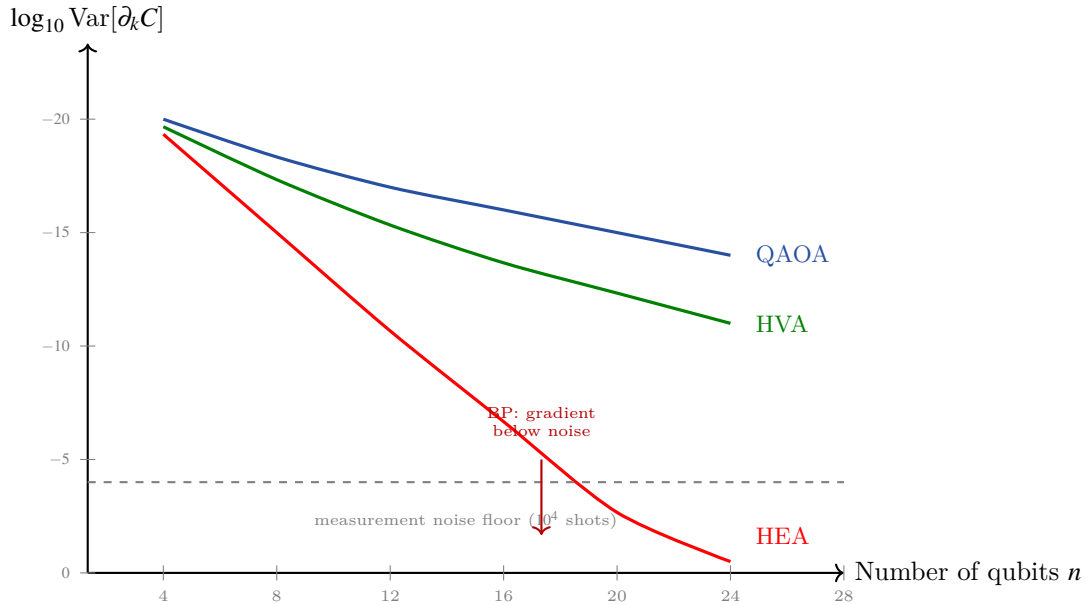
Ansatz	$\dim(\mathfrak{g})$	Barren Plateau?
HEA (deep, all-to-all)	$4^n - 1$	yes (exponential)
QAOA (p layers)	$O(n)$	no
HVA (1D Heisenberg)	$O(n^2)$	no
particle-conserving	$\binom{n}{k}^2 - 1$	depends on k
matchgate circuits	$O(n^2)$	no
single-qubit gates only	$3n$	no

Intuition. the DLA perspective gives a simple recipe: before training a circuit, compute (or estimate) the dimension of its lie algebra. if its polynomial in n , ur safe. if its exponential, ur in trouble. this can often be done analytically for structured ansatzes, or numerically for small instances.

11 Gradient Variance vs Number of Qubits

lets visualize the core phenomenon.

11.1 Exponential Decay of Gradient Variance



the plot shows the key difference: HEA gradient variance decays exponentially (straight line on log scale), while QAOA and HVA decay polynomially (much slower). the horizontal dashed line represents the measurement noise floor — below this, u cant distinguish the gradient from zero. for HEA, this happens around $n = 20$; for problem-inspired ansatze, the gradient stays well above the noise floor for much larger systems.

12 Practical Guidelines for Circuit Design

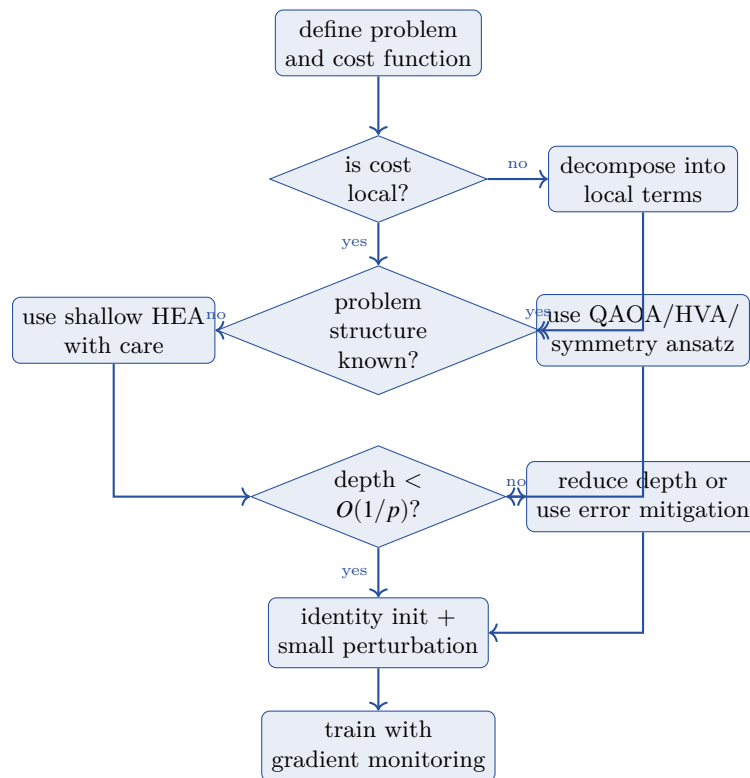
based on everything weve covered, here r concrete guidelines for designing trainable quantum circuits.

12.1 The Checklist

- G1. use local cost functions. decompose ur objective into a sum of terms, each acting on $O(1)$ qubits. avoid global observables like state fidelity unless absolutely necessary.
- G2. keep circuits shallow. target depth $L \in O(\log n)$ to $O(\text{poly}(\log n))$. if u need more depth, use layer-wise training to get there incrementally.
- G3. prefer problem-inspired ansatze. if ur problem has known structure (symmetries, locality, conservation laws), build them into the circuit. QAOA for optimization, HVA for physics, symmetry-preserving for chemistry.
- G4. initialize near identity. set initial parameters close to zero so $U(\theta_0) \approx \mathbb{I}$. add small random perturbations $\sim \mathcal{N}(0, 0.01)$ to break symmetry.

- G5. limit entanglement growth. dont use all-to-all connectivity unless needed. nearest-neighbor entangling gates create less entanglement per layer and delay the onset of BPs.
- G6. check the DLA dimension. if u can compute or estimate $\dim(\mathbf{g})$ for ur ansatz, do it. polynomial dimension = safe. exponential = redesign.
- G7. account for noise. circuit depth is bounded by $O(1/p)$ where p is the gate error rate. dont design circuits deeper than ur hardware can support.
- G8. monitor gradient norms during training. if gradients are shrinking as u scale up qubits, thats a red flag. plot $\text{Var}[\partial_k C]$ vs n to check for exponential decay.
- G9. use parameter correlations wisely. sharing parameters across layers (like in QAOA) reduces the effective parameter space and can improve trainability. but it also limits expressiveness.
- G10. consider alternatives to gradient descent. evolutionary strategies, bayesian optimization, and model-based methods dont rely on gradient information and can sometimes navigate barren plateaus. but they have their own scaling issues.

12.2 Decision Flowchart



13 Advanced Topics

13.1 Overparameterization and Lazy Training

larocca et al. also showed an interesting connection to classical deep learning: when a PQC has many more parameters than needed (overparameterized), it can enter a “lazy training” regime

where the parameters barely move from initialization. this avoids BPs but at the cost of the model essentially being linear around the initial point.

$$f(x; \theta) \approx f(x; \theta_0) + \nabla_{\theta} f(x; \theta_0)^T (\theta - \theta_0) \quad (37)$$

this is the quantum analog of the neural tangent kernel (NTK) regime. the model is effectively a kernel method with kernel:

$$k_{\text{NTK}}(x, x') = \nabla_{\theta} f(x; \theta_0)^T \nabla_{\theta} f(x'; \theta_0) \quad (38)$$

13.2 Quantum Natural Gradient

the quantum natural gradient (stokes et al. 2020) uses the fubini-study metric to define a geometry-aware gradient:

$$\theta_{t+1} = \theta_t - \eta F^{-1}(\theta_t) \nabla_{\theta} C(\theta_t) \quad (39)$$

where F is the quantum fisher information matrix:

$$F_{ij} = \text{Re} [\langle \partial_i \psi | \partial_j \psi \rangle - \langle \partial_i \psi | \psi \rangle \langle \psi | \partial_j \psi \rangle] \quad (40)$$

this can help with barren plateaus bcs it rescales the gradient by the curvature of the parameter space. in flat regions, F is small and F^{-1} amplifies the gradient. however, computing F requires $O(p^2)$ circuit evaluations for p parameters, which is expensive.

Key Point. quantum natural gradient doesnt solve barren plateaus — the gradient is still exponentially small. but F^{-1} can partially compensate, effectively converting an exponentially small gradient into a polynomially small effective gradient in some cases. its a mitigation, not a cure.

13.3 Error Mitigation and Trainability

error mitigation techniques (zero-noise extrapolation, probabilistic error cancellation) can partially address noise-induced BPs by reducing the effective noise rate. however, they come with their own overhead:

$$\text{cost of mitigation} \sim O\left(\frac{1}{q^{2L}}\right) \quad (41)$$

where $q = 1 - p$. this is exactly the same exponential factor that causes the noise-induced BP, so error mitigation and noise-induced BPs have the same scaling — u cant fully mitigate ur way out of a noise-induced BP.

14 Open Questions

barren plateaus r still an active research area. here r key open questions:

1. tight bounds: the current bounds on gradient variance r often loose. can we get tight (matching upper and lower) bounds for specific architectures?
2. higher-order methods: can second-order optimization (hessian-based) escape barren plateaus? the hessian itself might vanish exponentially, but theres ongoing work here

3. quantum advantage despite BPs: r there problems where the barren plateau threshold is higher for quantum models than for equivalent classical models? this would still give a relative advantage
4. adaptive circuits: circuits that change structure during training (adding/removing gates) might navigate the expressibility-trainability tradeoff dynamically. ADAPT-VQE is one example
5. measurement-based strategies: can mid-circuit measurements and classical feedforward help avoid BPs? early evidence suggests yes, but the theory is incomplete
6. non-gradient methods at scale: evolutionary strategies, reinforcement learning for parameter optimization, random search — can these scale better than gradient descent in BP regimes?

15 Summary

this was a dense lecture bcs barren plateaus r the make-or-break issue for variational QML. lets consolidate:

1. barren plateaus = gradient variance vanishes exponentially with system size. this makes optimization impossible with polynomial resources.
2. four mechanisms: expressibility (mcclean 2018), global cost functions (cerezo 2021), noise (wang 2021), entanglement (marrero 2021). they can act independently.
3. mcclean et al.: random circuits that form 2-designs have $\text{Var}[\partial_k C] \in O(2^{-n})$. this is the foundational result.
4. cerezo et al.: local cost functions save u — gradient variance depends on depth L , not width n . use local costs whenever possible.
5. noise-induced BPs: hardware noise independently kills trainability for circuits deeper than $O(1/p)$. this is a hard physical limit.
6. entanglement: too much entanglement scrambles gradient information. control entanglement growth through circuit design.
7. kernel concentration: barren plateaus in variational models correspond to kernel concentration in kernel models. same root cause (2-design convergence), same cure (restrict expressibility).
8. DLA dimension: the dimension of the dynamical lie algebra determines trainability. polynomial DLA = trainable, exponential DLA = barren plateau. this is the cleanest characterization we have.
9. mitigation strategies: local costs, shallow circuits, layer-wise training, identity initialization, problem-inspired ansatz. use all of them.
10. the fundamental tradeoff: expressibility vs trainability. u cant have both. the art is finding the sweet spot for ur specific problem.

Key Point. if u remember one thing from this lecture: more expressive is not better. the best quantum models r not the most general ones — they r the ones that r just expressive enough for the task while remaining trainable. this principle should guide every circuit design decision u make.

next lecture we'll look at concrete strategies for benchmarking and evaluating quantum models, including proper comparison with classical baselines.